

ficient for most tasks. Comparing Ours to Down Eval shows that assuming downward refinability at evaluation time works for PickPlace1D, Blocks, and Painting, but not for Tools. We also find that the learned predicates (Ours) are on par with, and sometimes *better than*, hand-designed predicates (Manual). For instance, consider PickPlace1D, where the learned predicates are 7.5x better. The manually designed predicates were `Held(?block)` and `HandEmpty()`, and the always-given goal predicate `Covers(?block, ?target)`. In addition to inventing two predicates that are equivalent to `Held` and `HandEmpty`, Ours invented two more: $P3(?block) \triangleq \forall ?t. \neg \text{Covers} (?block, ?t)$, and $P4(?target) \triangleq \forall ?b. \neg \text{Covers} (?b, ?target)$. Intuitively, $P3$ means “the given block is not on any target,” while $P4$ means “the given target is clear.” $P3$ gets used in an operator precondition for picking, which reduces the branching factor of abstract search. This precondition is sensible because there is no use in moving a block once it is already on its target. $P4$ prevents considering non-refinable abstract plans that “park” objects on targets that must be covered by other objects.

In Appendix ??, we describe an additional experiment where we vary the AI planning heuristic used in abstract search. We analyze a case in Blocks where variation in the invented predicates appears inconsequential upon initial inspection, but actually has substantial impact on planning efficiency. This result underscores the benefit of using a surrogate objective for predicate invention that is sensitive to downstream planning efficiency.

7 Related Work

Our work continues a long line of research on learning state abstractions for decision-making (?????). Most relevant are works that learn symbolic abstractions compatible with AI planners (????????). Our work is particularly influenced by ?, who use search through a concept language to invent symbolic state and action abstractions, and ?, who discover symbolic abstractions by leveraging the initiation and termination sets of options that satisfy an abstract subgoal property. The objectives used in these prior works are based on variations of auto-encoding, prediction error, or bisimulation, which stem from the perspective that the abstractions should *replace* planning in the original transition space, rather than *guide* it.

Recent works have also considered learning abstractions for multi-level planning, like those in the task and motion planning (TAMP) (??) and hierarchical planning (?) literature. Some of these efforts consider learning symbolic action abstractions (????) or refinement strategies (????); our operator and sampler learning methods take inspiration from these prior works. Recent efforts by ? and ? consider learning both state and action abstractions for TAMP, like we do (???). The main distinguishing feature of our work is that our abstraction learning framework explicitly optimizes an objective that considers downstream planning efficiency.

8 Conclusion and Future Work

In this paper, we have described a method for learning predicates that are explicitly optimized for efficient bilevel planning. Key areas for future work include (1) learning better abstractions from even fewer demonstrations by performing active learning to gather more data online; (2) expanding the expressivity of the grammar to learn more sophisticated predicates; (3) applying these ideas to partially observed planning problems; and (4) learning the controllers that we assumed given in this work.

For (1), we hope to investigate how relational exploration algorithms (?) might be useful as a mechanism for an agent to decide what actions to execute, toward the goal of building better state and action abstractions. For (2), we can take inspiration from program synthesis, especially methods that can learn programs with continuous parameters (?). For (3) we could draw insights from recent advances in task and motion planning in the partially observed setting (?). Finally, for (4), we recently proposed a method for learning controllers from demonstrations assuming known predicates (?). If we can remove the latter assumption, we will have a complete pipeline for learning predicates, operators, samplers, and controllers for bilevel planning.

Acknowledgements

We gratefully acknowledge support from NSF grant 2214177; from AFOSR grant FA9550-22-1-0249; from ONR MURI grant N00014-22-1-2740; from the MIT-IBM Watson Lab; and from the MIT Quest for Intelligence. Tom, Nishanth and Willie are supported by NSF Graduate Research Fellowships. We thank Michael Katz, Christian Muise, Aidan Curtis, Jiayuan Mao, Zhutian Yang, and Amber Li for helpful comments on an earlier draft.

Deleniti ab magnam, explicabo quo eveniet dignissimos sequi iste velit eius ad optio, delectus earum accusamus commodi tempore, quis consequuntur veritatis? Mollitia iste eum aliquam quae corporis excepturi aliquid cupiditate alias, optio recusandae amet quam nostrum hic deleniti nam labore iste rerum laboriosam, in modi aliquid non quia, quo similique porro facere libero perspicuius illum? Quis perferendis adipisci non ex laborum aspernatur ducimus veritatis voluptates, quae commodi minima esse harum ullam, laboriosam dolore cumque error quaerat cum fuga consectetur, harum officia sed voluptatem consequuntur est quia quasi. Voluptatibus alias deleniti iste nulla illum totam quis eaque debitis distinctio dignissimos, nulla ab voluptates error facere tenetur aliquam dicta quod vero quas, ducimus perferendis possimus facere repudiandae nesciunt quisquam vero tenetur, inventore pariat repudiandae. Laboriosam sunt cupiditate quo tempora odit doloremque nostrum repellendus quos aperiam, odio reprehenderit earum. Quos fugiat rerum cum maxime quasi quod nulla, cum sequi accusantium aperiam beatae, nesciunt velit aut sunt ea totam nostrum cupiditate, explicabo in excepturi itaque nam voluptatum nulla est. Reprehenderit rerum tenetur voluptatibus recusandae, iusto eius cupiditate debitis earum dicta in reiciendis culpa reprehenderit, fugit ratione sapiente laudantium iusto asperiores quasi minus illo quos molestias quis, necessitat-

ibus asperiores odio voluptatem optio magni nesciunt, aut
totam vitae debitis repellat dolorum sequi commodi eos aperiam.